CY Tech

Cycle ingénieur — Informatique — ING1 (GI)

Année universitaire 2025–2026

RAPPORT DE STAGE

Développement d'un système IA agentique pour la gestion des appels d'offres publics

OneTech Tender Agent

Réalisé par : **Khayem Ben Ghorbel**

Entreprise d'accueil : **One Tech Business Solutions (One Tech Group)**

Encadrant en entreprise : **Makram Ben Hamed**

Période : du 26 juin au 26 août 2026

Remerciements

Je tiens d'abord à remercier mon encadrant en entreprise, Makram Ben Hamed, pour la confiance accordée, la clarté du cadrage initial et la qualité de ses retours tout au long du stage. Ses orientations ont transformé un premier prototype de collecte en une plateforme complète, en fixant à chaque étape un cap exigeant mais atteignable.

Je remercie également les équipes de One Tech Business Solutions pour leur accueil et pour les échanges qui m'ont permis de comprendre la réalité métier du traitement des appels d'offres, bien au-delà de sa description théorique.

Je remercie enfin CY Tech, mon tuteur académique et l'ensemble des enseignants qui m'ont donné les bases scientifiques et techniques mobilisées dans ce projet.

Résumé

Ce rapport présente la conception et la réalisation d'un système d'intelligence artificielle agentique d'assistance à la gestion des appels d'offres publics, développé durant un stage de deux mois chez One Tech Business Solutions. Le point de départ est un problème métier concret : la veille sur les marchés publics est dispersée entre des listes, des pages de détail et des documents hétérogènes, et sa réalisation manuelle est longue, répétitive et exposée aux oublis.

La solution, nommée OneTech Tender Agent, articule un pipeline déterministe et une couche agentique. Un collecteur Python interroge les ressources publiques de la plateforme tunisienne HAICOP, normalise les données et les enregistre dans PostgreSQL avec des insertions idempotentes. Les documents publics sont téléchargés de manière bornée, identifiés par empreinte SHA-256, puis convertis en texte par extraction native, OCR Tesseract multilingue ou conversion LibreOffice. L'agent OpenClaw de l'entreprise consomme ensuite un contexte factuel empreinté pour produire une fiche d'analyse Go/No-Go en neuf sections, dont la cohérence est vérifiée avant toute persistance. L'automatisation quotidienne est orchestrée par n8n et diffusée par email et Telegram avec un registre anti-doublon, et un serveur MCP expose cinq outils bornés aux clients compatibles.

Le prototype validé centralise 100 appels d'offres et 152 documents intégralement traités, sans doublon observé pendant la campagne de validation. La campagne de validation compte 67 tests Python et 6 scripts SQL réussis, une preuve d'idempotence par double exécution réelle et un exercice complet de sauvegarde puis restauration vérifiée. Le système assiste la décision sans jamais s'y substituer : la validation finale reste humaine.

Mots-clés : IA agentique, appels d'offres publics, HAICOP, OpenClaw, PostgreSQL, Docker, n8n, OCR, MCP, Go/No-Go, idempotence, traçabilité.

Abstract

This report presents the design and implementation of an agentic artificial-intelligence system assisting the management of public tenders, developed during a two-month internship at One Tech Business Solutions. The starting point is a concrete business problem: monitoring public procurement is scattered across listings, detail pages and heterogeneous documents, making manual processing slow, repetitive and error-prone.

The resulting system, OneTech Tender Agent, combines a deterministic pipeline with an agentic layer. A Python collector queries the public resources of the Tunisian HAICOP platform, normalises the data and stores it in PostgreSQL through idempotent upserts. Public documents are downloaded under strict bounds, fingerprinted with SHA-256 and turned into text through native extraction, multilingual Tesseract OCR or LibreOffice conversion. The company's OpenClaw agent then consumes a fingerprinted factual context to produce a nine-section Go/No-Go analysis whose consistency is validated before persistence. Daily automation is orchestrated by n8n and delivered through email and Telegram with an anti-duplicate ledger, while an MCP server exposes five bounded tools to compatible clients.

The validated prototype centralises 100 tenders and 152 fully processed documents with zero duplicates. The validation campaign includes 67 passing Python tests and 6 SQL scripts, an idempotence proof through two identical real runs, and a complete backup-and-restore drill. The system assists the decision without replacing it: final validation remains human.

Keywords: agentic AI, public tenders, PostgreSQL, OpenClaw, n8n, OCR, MCP, automation, Go/No-Go, idempotence, traceability.

Table des matières

Remerciements 2

Résumé 3

Abstract 4

1. Introduction générale 7

2. Présentation de l'entreprise et cadre du stage 8

2.1 One Tech Business Solutions 8

2.2 Déroulement et encadrement du stage 8

2.3 Profil métier utilisé par le prototype 8

3. Contexte métier et problématique 9

3.1 Le traitement manuel d'un appel d'offres 9

3.2 L'écosystème tunisien : HAICOP et TUNEPS 9

3.3 Problématique 9

3.4 Objectifs 9

4. Fondements techniques et choix technologiques 11

4.1 Automatisation classique et approche agentique 11

4.2 OpenClaw 11

4.3 Model Context Protocol 11

4.4 PostgreSQL 11

4.5 Docker, Docker Compose et n8n 12

4.6 Traitement documentaire et OCR 12

4.7 Modèle local Phi-4 Mini 12

5. Analyse des besoins et spécification 13

5.1 Acteurs 13

5.2 Diagramme de cas d'utilisation 13

5.3 Cas d'utilisation détaillés 13

5.4 Règles métier du scoring 14

5.5 Critères d'acceptation 15

6. Conception et architecture 16

6.1 Vue d'ensemble 16

6.2 Répartition des responsabilités 16

6.3 Modèle de données 17

6.4 Séquence du pipeline quotidien 18

6.5 Séquence d'une analyse traçable 18

6.6 Déploiement 19

7. Réalisation 20

7.1 Organisation du code 20

7.2 Collecteur HAICOP 20

7.3 Persistance et idempotence 20

7.4 Traitement documentaire 20
7.5 Analyse Go/No-Go 21
7.6 Automatisation, notifications et interactions 21
7.7 Serveur MCP 21
7.8 Exploitation, sauvegarde et restauration 21
8. Validation, tests et résultats 22
8.1 Stratégie de validation 22
8.2 Résultats quantitatifs 22
8.3 Preuves d'idempotence 22
8.4 Cas d'analyse réels 22
8.5 Correction d'un faux positif 23
8.6 Évaluation du modèle local Phi-4 Mini 23
8.7 Résilience aux aléas externes 23
8.8 Sauvegarde et restauration vérifiées 24
9. Gestion du projet 25
9.1 Démarche incrémentale et jalons 25
9.2 Versionnement et discipline de dépôt 25
9.3 Gestion des risques 26
10. Qualité, sécurité et responsabilité 27
10.1 Moindre privilège 27
10.2 Gestion des secrets et traçabilité 27
10.3 Frontières légales et éthiques 27
10.4 Décision assistée, jamais automatisée 27
11. Compétences développées 28
11.1 Compétences techniques 28
11.2 Compétences transversales 28
12. Conclusion et apports du projet 30
Glossaire 31
Bibliographie et sitographie 32
Annexes 33
Annexe A — Commandes principales du système 33
Annexe B — Extraits SQL représentatifs 33
Annexe C — Matrice de validation finale 33

1. Introduction générale

Répondre à un marché public est une activité exigeante. Avant même d'écrire une proposition, une entreprise doit surveiller les publications, ouvrir chaque annonce, télécharger des documents aux formats variés, en extraire les dates, les conditions et les pièces exigées, puis décider si l'opportunité mérite l'effort d'une réponse. Ce travail est répétitif, chronophage et sensible aux oublis : une pièce administrative manquante ou une échéance mal lue suffisent à disqualifier un dossier.

Le sujet confié pour ce stage — développement d'un système IA agentique pour la gestion des appels d'offres publics — propose d'aborder ce problème autrement. L'objectif n'est pas de produire un résumé automatique de PDF, mais de transformer le processus en un workflow structuré, répétable et traçable, dans lequel un agent d'intelligence artificielle orchestre des outils spécialisés tandis que les opérations déterministes restent confiées à du code testé.

L'objectif pédagogique fixé par l'encadrant était double : comprendre en profondeur ce qu'est une IA agentique — et en particulier la distinction entre un agent et le modèle de langage qui l'anime — et maîtriser le fonctionnement d'OpenClaw, l'environnement agentique retenu. Le domaine des appels d'offres a servi de terrain d'application réaliste à cet apprentissage.

La démarche a été volontairement incrémentale. Un premier proof of concept a validé l'accès aux données publiques et l'analyse de trois dossiers réels. Chaque couche suivante — base de données, traitement documentaire, analyse persistée, automatisation, notifications, interopérabilité, exploitation — n'a été ajoutée qu'après validation de la précédente, avec des critères d'acceptation définis à l'avance.

Ce rapport suit cette progression. Il présente d'abord l'entreprise et le contexte métier, puis la problématique et les objectifs. Il expose ensuite les fondements techniques et les choix technologiques, l'analyse des besoins, la conception, la réalisation et la validation du système. Il se conclut par la gestion du projet, les aspects de qualité et de sécurité, les compétences développées et les apports du projet.

2. Présentation de l'entreprise et cadre du stage

2.1 One Tech Business Solutions

One Tech Business Solutions est l'entité de services numériques du groupe tunisien One Tech Group. Son activité couvre le développement logiciel, l'intégration de systèmes et de systèmes d'information, l'intelligence artificielle et la Data, l'automatisation ainsi que les projets de transformation digitale. Ses clients sont des organisations publiques et privées pour lesquelles elle conçoit, intègre et maintient des solutions numériques.

Pour une entreprise de ce profil, les marchés publics représentent un canal commercial naturel : une part significative des projets numériques d'envergure passe par des appels d'offres. Encore faut-il identifier, parmi le flux quotidien de publications, les opportunités qui correspondent réellement au périmètre technique de l'entreprise — et le faire assez tôt pour laisser le temps de préparer une réponse sérieuse.

2.2 Déroulement et encadrement du stage

Le stage s'est déroulé du 26 juin au 26 août 2026, à temps plein, sous l'encadrement de Makram Ben Hamed. Le suivi a reposé sur un cadrage initial précis, des démonstrations d'étape et des retours réguliers : le premier prototype présenté début juillet a reçu un retour positif accompagné d'axes d'exploration — persistance en base de données, diffusion quotidienne, canaux de messagerie, orchestration et interopérabilité — qui ont structuré la suite du travail.

Une exigence de l'encadrement mérite d'être soulignée : l'objectif n'était pas d'empiler des fonctionnalités, mais de comprendre et de démontrer chaque mécanisme. Cette exigence a orienté des choix décrits plus loin, comme la preuve d'idempotence par double exécution ou la validation stricte des fiches d'analyse avant persistance.

2.3 Profil métier utilisé par le prototype

Le moteur de présélection s'appuie sur un profil métier de l'entreprise, validé en version V1 pour le périmètre du prototype. Ce profil favorise les opportunités relevant du logiciel, des applications web, des plateformes numériques, de l'intégration de systèmes d'information, de l'IA, de la Data et de l'automatisation ; il pénalise les dossiers manifestement hors périmètre, comme les travaux publics, les fournitures alimentaires, le matériel médical ou les achats purement matériels sans composante logicielle. Ce référentiel borne volontairement ce que le système peut affirmer : il ne prête à l'entreprise aucune capacité qui n'y figure pas.

3. Contexte métier et problématique

3.1 Le traitement manuel d'un appel d'offres

Le traitement complet d'un appel d'offres enchaîne une douzaine d'étapes : réception ou découverte de l'annonce, lecture des documents, extraction des informations clés, analyse des exigences techniques puis administratives, vérification des critères d'éligibilité, repérage des échéances, évaluation des risques, estimation de l'adéquation avec les compétences internes, décision de réponse, préparation d'une checklist et suivi jusqu'au dépôt. Chaque étape est simple isolément ; c'est leur répétition sur des dizaines de dossiers, sous contrainte de délai, qui rend le processus coûteux et fragile.

Trois caractéristiques compliquent son automatisation. D'abord, l'information est dispersée : la liste des publications, la page de détail et les pièces jointes ne racontent chacune qu'une partie du dossier. Ensuite, les documents sont hétérogènes : PDF avec ou sans couche texte, DOCX, anciens DOC binaires, voire images. Enfin, un titre est trompeur : une annonce au vocabulaire informatique peut masquer un achat purement matériel, et inversement.

3.2 L'écosystème tunisien : HAICOP et TUNEPS

En Tunisie, la publication des appels d'offres publics passe par le portail de la HAICOP (marchespublics.gov.tn), qui expose les annonces, les pages de détail et une partie des documents. La soumission des offres et certains dossiers complets relèvent de la plateforme TUNEPS, qui exige un compte fournisseur authentifié. Cette séparation a structuré une frontière du projet, posée dès le départ : le système n'exploite que les ressources publiques et ne contourne aucune authentification. Lorsqu'un document relève d'un espace authentifié, il est demandé à un humain disposant des droits.

3.3 Problématique

Comment concevoir un système agentique capable de collecter de manière fiable des appels d'offres publics, d'en traiter les documents hétérogènes, d'en évaluer la pertinence pour l'entreprise et de diffuser un résultat traçable, sans doublon, sans exposition de secrets et sans automatiser abusivement la décision finale ?

Cette question en contient plusieurs : la fiabilité de la collecte face aux aléas réseau ; l'idempotence de toutes les opérations rejouables ; la maîtrise des risques liés au téléchargement de fichiers tiers ; la traçabilité d'une analyse produite par un modèle de langage ; la prévention des doubles envois ; et la juste place de l'humain dans une décision à impact commercial.

3.4 Objectifs

Objectifs fonctionnels

- collecter les appels d'offres publics récents de la plateforme HAICOP ;
- normaliser et persister les données dans PostgreSQL, sans perte du contenu original ;
- télécharger, identifier et extraire le texte des documents associés ;
- présélectionner les dossiers cohérents avec le profil métier de l'entreprise ;

- produire une fiche d'analyse structurée et une recommandation Go/No-Go traçable ;
- générer un digest quotidien et le diffuser par email et Telegram ;
- répondre aux sollicitations des membres du groupe Telegram dédié ;
- exposer les capacités principales à des clients compatibles via MCP.

Objectifs non fonctionnels

- idempotence des collectes, des traitements documentaires, des analyses et des diffusions ;
- traçabilité : conservation des données brutes, empreintes SHA-256, séparation entre faits et raisonnement ;
- sécurité : secrets hors du dépôt, services liés à l'interface locale, surfaces d'attaque minimales ;
- robustesse : reprise après erreur, statuts observables, journalisation des incidents ;
- reproductibilité : infrastructure décrite par Docker Compose et migrations SQL versionnées ;
- défense contre l'injection d'instructions : les textes d'appels d'offres sont des données, jamais des ordres.

4. Fondements techniques et choix technologiques

4.1 Automatisation classique et approche agentique

Une automatisation classique enchaîne des étapes entièrement prédéfinies : lire une source, appliquer des règles, produire un résultat. Elle excelle sur les tâches déterministes mais devient rigide dès qu'il faut interpréter des documents variés. À l'inverse, un modèle de langage seul interprète très bien un texte, mais n'offre par lui-même ni mémoire d'état, ni outils, ni garantie de reproductibilité.

Un système agentique combine les deux. Il faut pour cela distinguer quatre notions souvent confondues :

- **le modèle**, qui génère et interprète du langage ;
- **l'agent**, qui encadre le modèle : il reçoit un objectif, dispose d'un contexte et choisit parmi des outils autorisés ;
- **les outils**, qui exécutent les opérations fiables — une requête SQL, un téléchargement, un calcul d'empreinte ;
- **le workflow**, qui borne l'ordre et le périmètre des actions possibles.

Le principe directeur du projet découle de cette distinction : tout ce qui peut être codé et testé est codé et testé ; le modèle n'intervient que là où l'interprétation sémantique est réellement nécessaire. Cette répartition, détaillée au chapitre 6, rend le système plus sûr et plus auditable qu'un agent monolithique à qui l'on confierait toutes les opérations.

4.2 OpenClaw

OpenClaw est l'environnement agentique retenu. Il organise un espace de travail persistant autour de fichiers de gouvernance (règles permanentes, identité, mémoire) et de skills : des capacités décrites en langage naturel, munies d'un descripteur de déclenchement et d'une procédure. Deux skills ont été développées : collecte-haicop-ao, qui encadre la recherche et la collecte d'appels d'offres, et analyse-ao, qui définit la procédure d'analyse et le format exact de la fiche produite.

Une instance dédiée à l'entreprise, distincte de toute instance personnelle, a été créée dès le début du projet : espace de travail propre, configuration propre, passerelle locale dédiée (127.0.0.1:18790) et règles de confidentialité explicites. Cette séparation garantit qu'aucune donnée personnelle ne se mêle au contexte de l'entreprise, et réciproquement.

4.3 Model Context Protocol

Le Model Context Protocol (MCP) standardise l'exposition d'outils à des agents et applications d'IA. Son intérêt est le découplage : plutôt que d'écrire une intégration par client, le projet expose une seule fois cinq opérations bornées — collecte, liste des dossiers ouverts, détail d'un dossier, analyse, génération de digest — que tout client compatible peut découvrir et invoquer. Les entrées sont validées, les sorties limitées en taille et les messages d'erreur expurgés de tout secret.

4.4 PostgreSQL

PostgreSQL 16 constitue la source de vérité du système. Trois propriétés ont motivé ce choix : les transactions et contraintes d'intégrité, indispensables à l'idempotence ; le type JSONB, qui permet de conserver les réponses HAICOP originales sans perte tout en modélisant proprement les champs exploités ; et la richesse de l'écosystème (pgAdmin pour l'administration, sauvegarde et restauration outillées).

4.5 Docker, Docker Compose et n8n

L'infrastructure est décrite par Docker Compose : PostgreSQL, pgAdmin, Mailpit (boîte email locale de test), le worker applicatif et n8n. Les volumes persistants survivent à la recréation des conteneurs et tous les ports sont liés à 127.0.0.1. n8n, orchestrateur visuel de workflows, assure la planification : il déclenche le pipeline quotidien les jours ouvrés à 08:00 (fuseau Europe/Paris) et conserve l'historique des exécutions. Il ne remplace ni Python ni OpenClaw : il décide du quand, jamais du comment.

4.6 Traitement documentaire et OCR

L'extraction native couvre les PDF disposant d'une couche texte et les DOCX. Pour les documents scannés, Poppler rend les pages en images et Tesseract effectue une reconnaissance optique en français, en arabe et en anglais — trois langues effectivement rencontrées dans les dossiers tunisiens. Les anciens fichiers DOC binaires sont convertis par LibreOffice en mode headless, dans un profil temporaire isolé et sous timeout. Toutes les étapes sont bornées en taille, en durée et en nombre de pages.

4.7 Modèle local Phi-4 Mini

À la demande de l'encadrant, un petit modèle de Microsoft, Phi-4 Mini, a été évalué localement via Ollama, sur CPU. L'enjeu était de mesurer ce qu'un modèle léger et souverain peut apporter à ce type de système — l'évaluation complète est présentée au chapitre 8.

5. Analyse des besoins et spécification

5.1 Acteurs

Acteur	Rôle et besoin principal
Collaborateur One Tech	Recevoir une veille quotidienne concise et interroger l'agent depuis le groupe Telegram
Analyste / responsable	Comprendre le score, les risques et les sources d'une analyse ; garder la décision finale
Administrateur	Superviser les services, consulter les journaux, sauvegarder et restaurer
n8n (système)	Déclencher des traitements idempotents selon un planning
HAICOP (système)	Fournir les annonces, pages de détail et documents publics

5.2 Diagramme de cas d'utilisation

La figure 1 synthétise les interactions entre les acteurs et le système. Le cas d'utilisation central, la génération du digest quotidien (UC4), inclut la collecte, le traitement documentaire et l'analyse : c'est l'enchaînement complet que l'automatisation rejoue chaque jour ouvré.

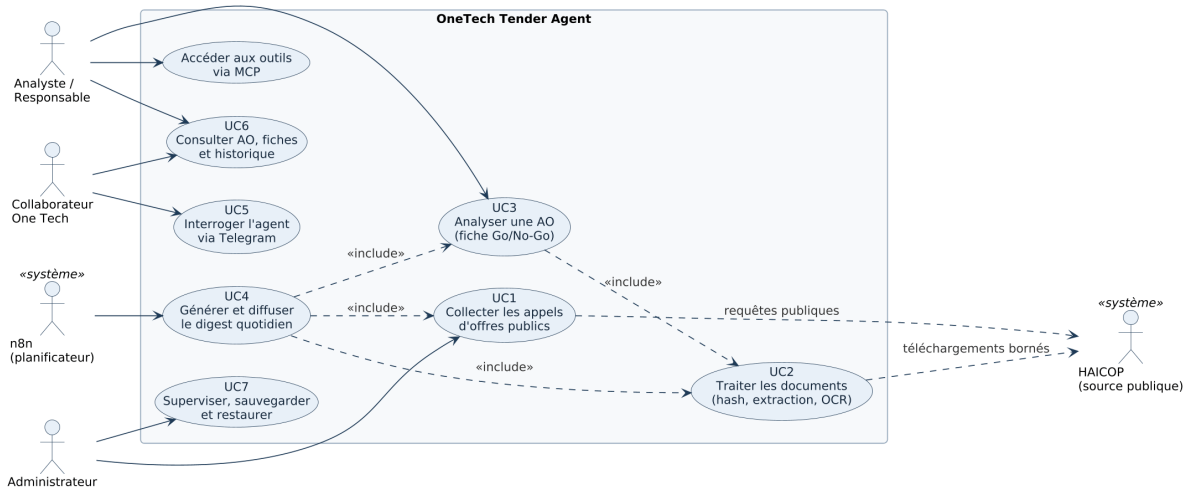


Figure 1 — Diagramme de cas d'utilisation du système

5.3 Cas d'utilisation détaillés

UC1 — Collecter les appels d'offres

1. L'utilisateur, n8n ou un client MCP demande une collecte, éventuellement bornée (fenêtre de fraîcheur, limite).
2. Le collecteur ouvre un journal d'exécution au statut en_cours.
3. Il interroge les ressources publiques HAICOP avec pagination, délais d'attente et reprises.
4. Il normalise chaque annonce (identifiant, organisme, titre, dates) et conserve la réponse brute en JSONB.

5. Il réalise des UPSERT : insertion si l'annonce est nouvelle, mise à jour sinon — aucun doublon observé lors des rejouements de validation.
6. Il clôture le journal avec les compteurs, ou avec l'erreur rencontrée.

UC2 — Traiter les documents

1. Le système sélectionne les documents en attente de traitement.
2. Il vérifie que l'URL appartient au domaine et au chemin publics autorisés.
3. Il télécharge le fichier sous limites de taille et de redirections, puis détecte son type réel.
4. Il calcule l'empreinte SHA-256 et stocke le fichier localement.
5. Il extrait le texte — extraction native, OCR ou conversion selon le format.
6. Il enregistre le statut, le texte et l'éventuelle erreur ; les relances ignorent les documents déjà traités.

UC3 — Analyser un appel d'offres

1. Le système exporte depuis PostgreSQL un contexte factuel complet : faits officiels, champs de détail, profil métier et texte intégral des documents.
2. Une empreinte SHA-256 déterministe du contexte canonique est calculée.
3. OpenClaw lit l'ensemble des sources et rédige une fiche en neuf sections, avec quatre scores et une décision.
4. La commande d'enregistrement vérifie la complétude de la fiche, la cohérence scores/décision et l'intégrité de l'empreinte.
5. L'analyse est persistée de manière idempotente : rejouer le même contexte réutilise le même enregistrement.

UC4 — Générer et diffuser le digest quotidien

1. n8n appelle le worker authentifié, qui confie une mission bornée à l'agent de l'entreprise.
2. Le pipeline collecte, traite les documents et analyse les candidats retenus par la présélection.
3. Un digest Markdown est généré, hashé et enregistré.
4. Pour chaque canal et destinataire, une preuve de diffusion est réservée avant l'envoi : un digest déjà transmis n'est jamais renvoyé.

UC5 — Interroger l'agent via Telegram

1. n8n interroge les mises à jour Telegram depuis un offset durable.
2. Seuls les commandes, mentions du bot et réponses à ses messages sont retenus ; les conversations ordinaires du groupe sont ignorées.
3. Le contenu est nettoyé puis transmis au worker, et la réponse de l'agent est publiée dans le groupe.
4. Chaque interaction est auditée par empreinte, avec des contraintes d'unicité qui empêchent toute double réponse.

5.4 Règles métier du scoring

La fiche Go/No-Go note quatre axes sur 25 points chacun : l'adéquation avec le profil métier, la faisabilité du délai, la charge administrative et le niveau de risque. Le total sur 100 détermine la recommandation : GO à partir de 70, À DISCUTER à partir de 45, NO_GO en dessous. Deux garde-fous complètent le barème : une analyse partielle — lorsqu'une pièce du dossier manque — ne peut pas produire un GO définitif, et la présélection est autorisée à ne retenir aucune opportunité plutôt que de forcer un faux positif.

5.5 Critères d'acceptation

- deux collectes identiques ne créent aucun doublon ;
- un document déjà traité n'est pas retéléchargé lors d'une relance normale ;
- une analyse rejouée sur un contexte inchangé réutilise son enregistrement ;
- un digest rejoué n'est jamais renvoyé au même destinataire sur le même canal ;
- les messages d'erreur ne révèlent aucun secret ;
- la restauration d'une sauvegarde est vérifiée par comparaison des compteurs dans une base isolée ;
- l'ensemble démarre et se contrôle par des scripts documentés.

6. Conception et architecture

6.1 Vue d'ensemble

L'architecture, présentée en figure 2, superpose quatre couches. La couche d'acquisition (Python) collecte les annonces et traite les documents. PostgreSQL, au centre, conserve les faits officiels, les preuves documentaires, les analyses et les registres de diffusion. La couche d'intelligence — l'agent OpenClaw de l'entreprise, le serveur MCP et, en second avis, le modèle local — consomme et produit du contenu à partir de cette base. Enfin, la couche d'orchestration et de diffusion (n8n, worker, canaux) anime l'ensemble au quotidien.

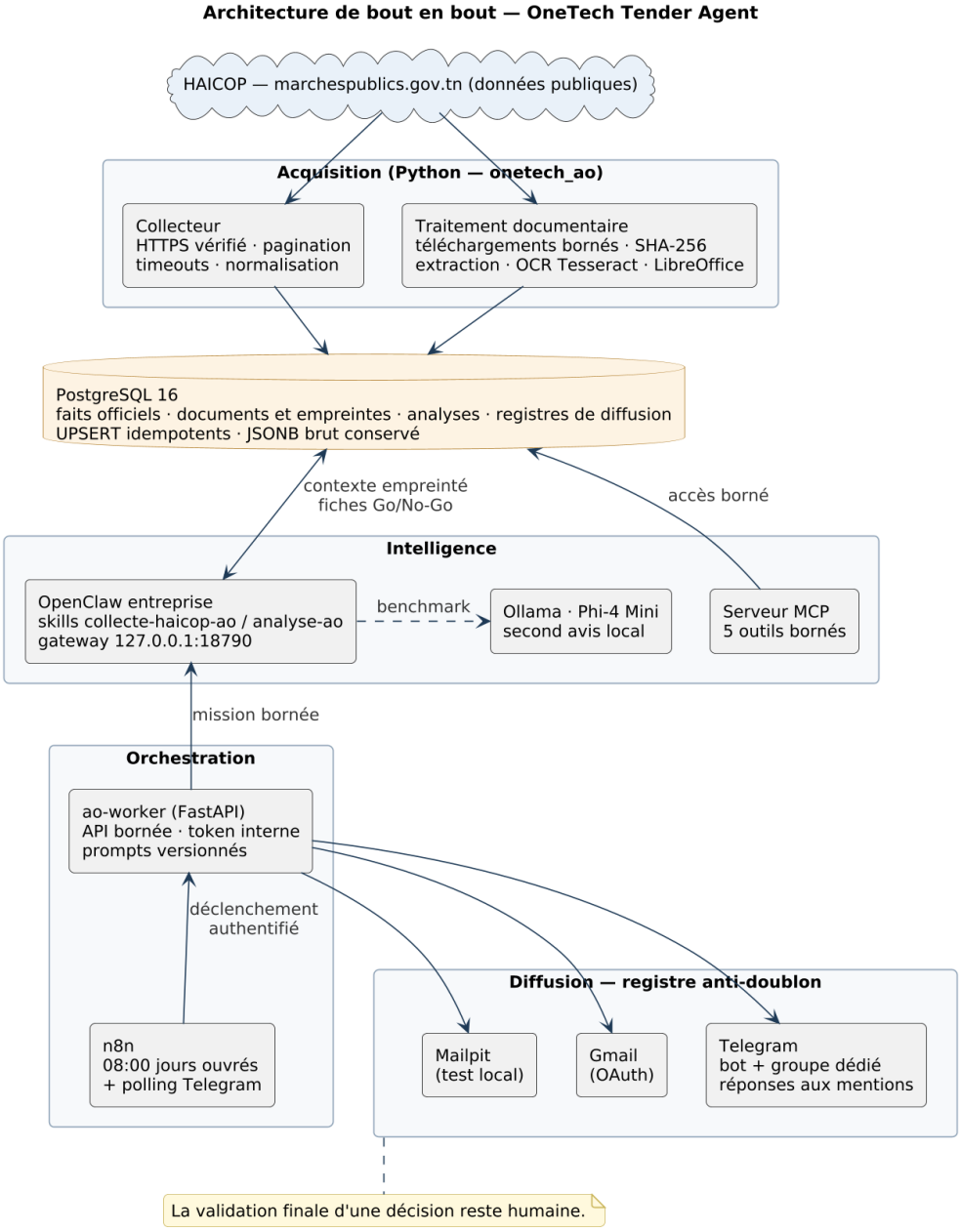


Figure 2 — Architecture de bout en bout du système

6.2 Répartition des responsabilités

Composant	Responsabilité	Ce qu'il ne fait jamais
Python (onetech_ao)	Opérations déterministes : HTTP, parsing, SQL, hash, fichiers, envois	Interpréter le sens d'un document
PostgreSQL	État, intégrité, contraintes, transactions, audit	Exécuter de la logique métier applicative
OpenClaw	Lecture sémantique, choix de skill, rédaction des fiches	Modifier la base sans passer par la CLI validante
n8n	Planification, déclenchement, historique	Envoyer un prompt arbitraire au modèle
Serveur MCP	Interface standard et bornée vers les capacités	Créer silencieusement une décision
Humain	Validation finale de toute décision engageante	—

Cette matrice est le cœur de la conception. Elle traduit une conviction acquise pendant le stage : la valeur d'un système agentique ne vient pas de ce que l'on confie au modèle, mais de la précision avec laquelle on délimite ce qu'on ne lui confie pas.

6.3 Modèle de données

Le schéma relationnel, versionné en sept migrations SQL idempotentes, sépare strictement quatre familles d'information : les faits officiels issus de HAICOP, les preuves documentaires, le raisonnement généré par l'agent et les registres d'automatisation et de diffusion. La figure 3 en donne la vue d'ensemble.

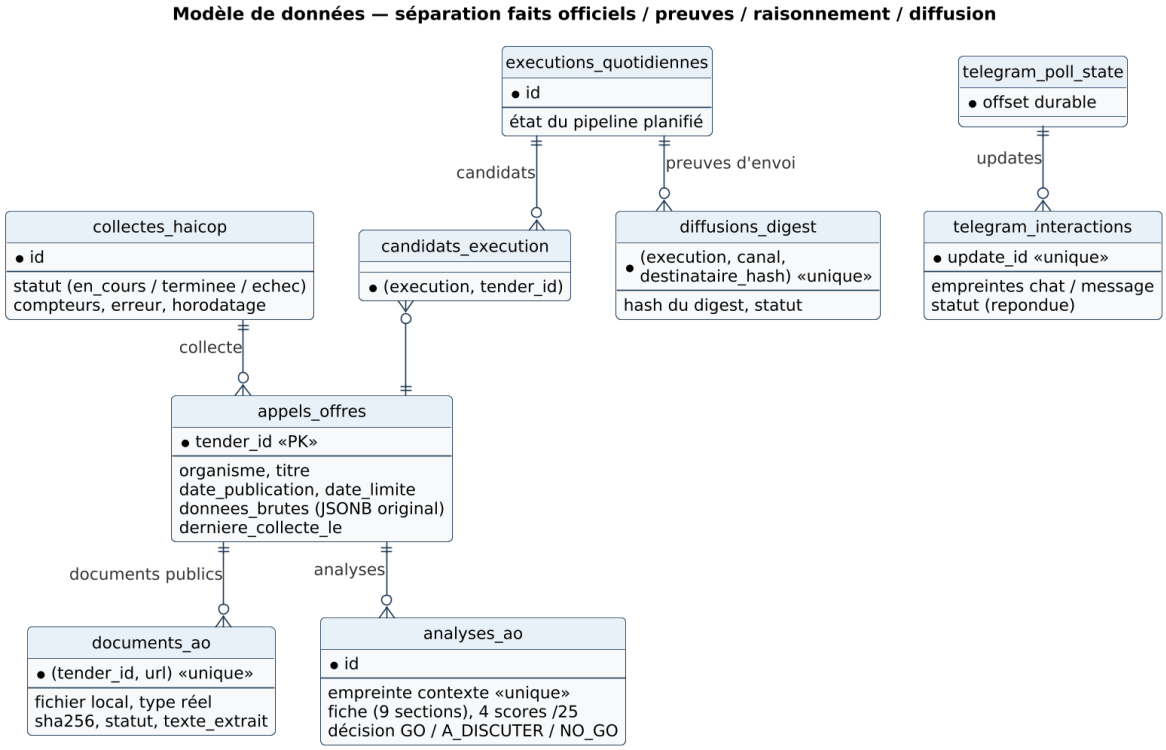


Figure 3 — Modèle de données et contraintes d'unicité

L'idempotence est portée par les contraintes elles-mêmes : clé primaire sur l'identifiant d'appel d'offres, unicité du couple (dossier, URL) pour les documents, unicité de l'empreinte de contexte pour les analyses, unicité du triplet (exécution, canal, destinataire) pour les diffusions et unicité des identifiants de mise à jour Telegram. Rejouer une opération met à jour l'existant au lieu de le dupliquer — la propriété est garantie par la base, pas seulement par la discipline du code.

6.4 Séquence du pipeline quotidien

La figure 4 déroule l'exécution planifiée. Deux points méritent l'attention : le worker constitue une frontière de sécurité — n8n ne peut invoquer que des routes métier définies, avec un jeton interne, et les prompts transmis à l'agent sont versionnés, jamais arbitraires — et la boucle de diffusion réserve une preuve d'envoi avant chaque transmission, ce qui a empêché tout renvoi doublonné pendant la campagne de validation.

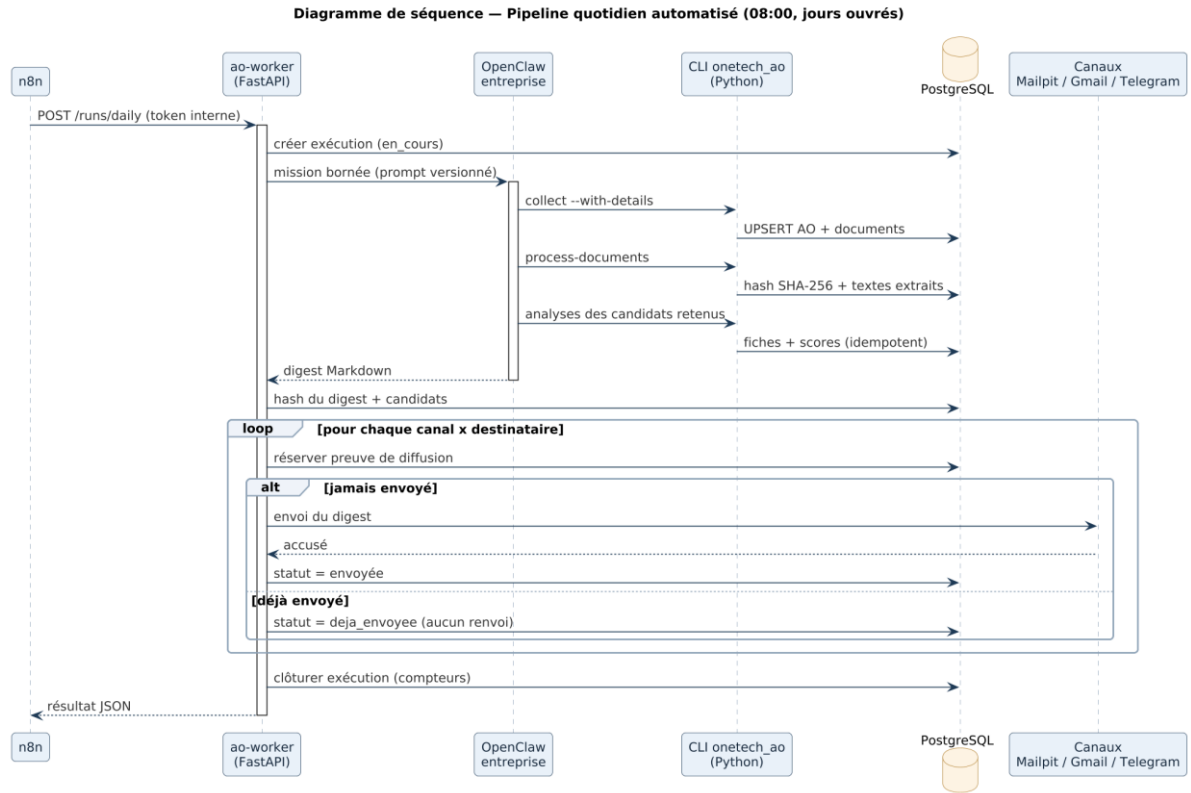


Figure 4 — Diagramme de séquence — pipeline quotidien automatisé

6.5 Séquence d'une analyse traçable

La figure 5 détaille le mécanisme central du projet : l'analyse Go/No-Go. Le contexte factuel est exporté depuis la base et empreinté avant lecture ; la fiche produite est validée dans son intégralité avant persistance ; et un contexte modifié entre la lecture et l'enregistrement provoque un refus. La décision est ainsi liée cryptographiquement à l'état exact des sources qui l'ont fondée.

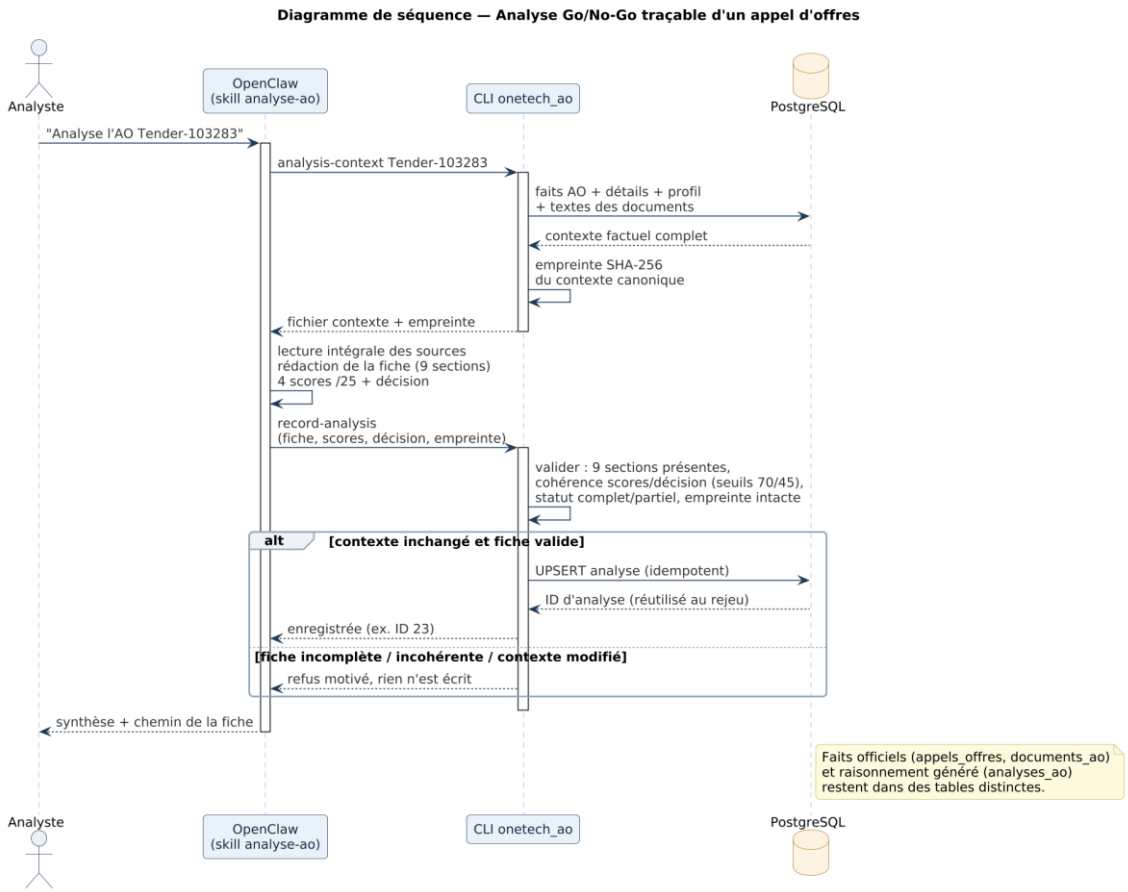


Figure 5 — Diagramme de séquence — analyse Go/No-Go et validation avant persistance

6.6 Déploiement

Service	Rôle	Exposition
postgres	Source de vérité (PostgreSQL 16)	127.0.0.1:5432, volume persistant
pgadmin	Administration graphique	127.0.0.1:5050
mailpit	Boîte email locale de test	127.0.0.1:8025
ao-worker	API FastAPI bornée vers OpenClaw	Réseau Docker interne uniquement
n8n	Planification et historique des workflows	127.0.0.1:5678
OpenClaw entreprise	Agent et skills, sur l'hôte	Passerelle 127.0.0.1:18790

Aucun service n'est exposé au réseau : l'ensemble est lié aux interfaces locales, et le worker n'est joignable que depuis le réseau Docker interne. Les secrets — jetons, clés OAuth, identifiants — sont lus depuis un fichier d'environnement ignoré par Git et documenté par un gabarit d'exemple.

7. Réalisation

7.1 Organisation du code

Le package Python onetech_ao est découpé par responsabilités, ce qui permet de tester chaque brique isolément :

```
src/onetech_ao/
├── config.py      configuration et variables d'environnement
├── haicop.py      client HTTP, normalisation, parsing des pages de détail
├── collector.py   orchestration de la collecte et journalisation
├── database.py    accès PostgreSQL, transactions, UPSERT
├── documents.py   téléchargement borné, SHA-256, extraction, OCR, conversion
├── analysis.py    contexte empreinté, validation et persistance des fiches
├── automation.py  exécutions quotidiennes, digest, registres de diffusion
├── telegram.py    client Telegram, filtrage des interactions
├── worker.py      API FastAPI bornée exposée à n8n
├── mcp_server.py  serveur MCP : cinq outils bornés
└── cli.py         interface en ligne de commande à sorties JSON
```

Toutes les commandes de la CLI renvoient du JSON encodé en UTF-8, directement exploitable par l'agent : la même interface sert à l'humain, à OpenClaw, au worker et aux tests.

7.2 Collecteur HAICOP

La liste publique des appels d'offres n'apparaît pas dans le HTML statique du portail : elle est chargée dynamiquement. L'analyse du code de la page a révélé que la même adresse sert d'endpoint JSON paginé lorsqu'elle est interrogée avec les paramètres et en-têtes appropriés. Le collecteur exploite ce mécanisme public de façon respectueuse : requêtes HTTPS vérifiées, User-Agent explicite, délais d'attente, reprises bornées et arrêt de la pagination dès que la fenêtre de fraîcheur demandée est couverte.

Chaque annonce est normalisée — identifiants, dates, textes, booléens — et la réponse originale est conservée intégralement en JSONB : aucun champ n'est perdu, même s'il n'est pas encore modélisé. Avec l'option d'enrichissement, le collecteur visite les pages de détail publiques et ne retient, parmi les liens de documents, que ceux qui correspondent strictement au chemin public attendu.

7.3 Persistance et idempotence

Les écritures utilisent la construction INSERT ... ON CONFLICT DO UPDATE de PostgreSQL : une annonce nouvelle est insérée, une annonce connue est mise à jour et sa date de dernière collecte actualisée. Les valeurs passent par des paramètres liés — jamais par concaténation dans le SQL — et chaque lot est transactionnel : une erreur grave annule proprement l'ensemble plutôt que de laisser un état partiel.

7.4 Traitement documentaire

Le processeur documentaire applique une chaîne de contrôles avant, pendant et après chaque téléchargement : validation stricte de l'URL, limites de taille et de redirections, détection du type réel à partir du contenu et non de l'extension, contrôle de sûreté des archives DOCX, empreinte SHA-256, puis extraction adaptée au format — native, OCR Tesseract (français, arabe, anglais) après rendu

Poppler pour les scans, ou conversion LibreOffice isolée pour les anciens DOC. Chaque document porte un statut observable ; les relances ignorent les documents traités, peuvent retenter les erreurs et permettent un retraitement volontaire après amélioration d'un parseur.

7.5 Analyse Go/No-Go

La commande d'export de contexte assemble, pour un dossier donné, tout ce que la base sait : faits officiels, champs de détail, statuts documentaires, texte intégral de chaque pièce avec repères de pages, et profil métier. Le fichier produit est empreinté ; les éléments purement opérationnels (horodatages de collecte, chemins locaux) sont exclus du calcul afin que l'empreinte ne varie qu'avec le fond.

La skill analyse-ao impose ensuite une discipline stricte à l'agent : lecture de l'intégralité des sources, fiche en neuf sections dans un ordre fixe, mention explicite « Non trouvé » pour toute information absente, signalement d'une analyse partielle, quatre notes justifiées et une décision. À l'enregistrement, la CLI rejoue toutes les vérifications : sections présentes, cohérence entre les notes et la décision au regard des seuils, statut de complétude, intégrité de l'empreinte. Une fiche incomplète, incohérente ou fondée sur un contexte modifié est refusée — rien n'est écrit.

7.6 Automatisation, notifications et interactions

Le workflow n8n quotidien appelle le worker, qui confie à l'agent une mission bornée par des prompts versionnés. Le digest produit est hashé, enregistré, puis diffusé : Mailpit sert de canal de test local sans accès Internet, Gmail assure l'envoi réel multi-destinataires via OAuth — sans stockage de mot de passe — et Telegram publie dans un groupe dédié, en fragments compatibles avec les limites de la plateforme. Un second workflow interroge Telegram en continu et ne route vers l'agent que les commandes, mentions et réponses au bot ; les identifiants et contenus sont audités sous forme d'empreintes, et des contraintes d'unicité bloquent les doubles réponses pour une même clé de diffusion.

7.7 Serveur MCP

Le serveur MCP expose en local cinq outils — collecte, liste des dossiers ouverts, détail, analyse, digest — qui réutilisent exactement les mêmes classes Python et la même base que la CLI. Les entrées sont validées, les sorties bornées en taille et les erreurs expurgées. L'outil d'analyse lit une analyse persistée ou prépare le contexte : il ne fabrique jamais silencieusement une décision.

7.8 Exploitation, sauvegarde et restauration

Des scripts d'exploitation couvrent le cycle de vie complet : démarrage de la pile, application des migrations, activation idempotente des workflows, contrôle de santé à la demande. Le démarrage est volontairement manuel et documenté. La sauvegarde assemble le dump PostgreSQL, l'état n8n, les workflows, les documents, les fiches et une configuration assainie — sans aucun secret — le tout accompagné d'un manifeste et de sommes de contrôle SHA-256. Un exercice de restauration complet est intégré : la sauvegarde est restaurée dans une base isolée, les compteurs de chaque table sont comparés, puis la base temporaire est supprimée.

8. Validation, tests et résultats

8.1 Stratégie de validation

La validation combine quatre niveaux complémentaires : des tests unitaires Python (normalisation, parsing, sécurité des URL, extraction, scoring, Telegram, MCP) ; des tests d'intégration PostgreSQL auto-nettoyants (transactions, UPSERT, idempotence) ; des smoke tests SQL exécutés dans des transactions annulées (contraintes, vues, cohérence des analyses et des diffusions) ; et des validations de bout en bout sur données réelles — collectes effectives, documents réels, analyses menées dans l'interface de l'agent, diffusions Gmail et Telegram, outils MCP et exercice de restauration.

8.2 Résultats quantitatifs

Indicateur	Résultat validé
Appels d'offres centralisés	100
Documents enregistrés	152 — 41 PDF, 74 DOCX, 35 DOC, 1 PNG, 1 JPEG
Fichiers présents / hashés / texte extrait	152 / 152 / 152
Doublons (appels d'offres et documents)	0
Documents en attente ou en échec	0
Tests Python	67 / 67 réussis
Scripts de test SQL	6 / 6 réussis
Outils MCP découverts et exécutés	5 / 5
Canaux de diffusion validés	Mailpit, Gmail, Telegram

8.3 Preuves d'idempotence

La propriété d'idempotence a été prouvée par l'expérience, pas seulement affirmée par la conception. Lors de la mise en service du collecteur, deux collectes réelles identiques et enrichies ont été exécutées : la première a inséré 20 appels d'offres et 30 documents, la seconde a mis à jour exactement les mêmes lignes sans en créer aucune. Le même protocole a été appliqué aux autres étages : un retraitement documentaire à vide (zéro document en attente), le rejeu d'une analyse sur contexte inchangé (réutilisation de l'identifiant existant) et le rejeu d'une diffusion (statut « déjà envoyée », aucun second message).

8.4 Cas d'analyse réels

Dossier	Organisme	Scores (adéq./délai/adm./risque)	Total	Décision
Tender-103283	Société Nationale Immobilière (SNIT Centre)	0 / 20 / 1 / 3	24 / 100	NO_GO — analyse ID 23

Tender-103307	Hôpital Charles Nicolle de Tunis	—	36 / 100	NO_GO — analyse ID 53

Ces deux dossiers, dont l'intégralité des pièces a été extraite, illustrent la fonction première du système : écarter avec des justifications sourcées des opportunités lisibles mais hors du périmètre logiciel de l'entreprise. Sur la fenêtre de collecte du prototype, aucune opportunité n'a atteint les seuils positifs — un comportement conforme à la règle qui autorise le digest à ne rien retenir plutôt qu'à forcer un faux positif. La validation du chemin positif repose, à ce stade, sur les tests de cohérence scores/décision ; produire des cas réels de GO, en calibrant le barème sur des décisions historiques de l'entreprise, constitue la première perspective du chapitre 12.

8.5 Correction d'un faux positif

Un incident de présélection a marqué le projet. Une annonce portant sur des équipements de formation au gaz avait été retenue par un filtre trop permissif, et un test technique lui avait attribué un score manifestement erroné. L'analyse de l'incident a conduit à durcir le filtre : recherche de mots entiers, reconnaissance explicite du vocabulaire des réseaux informatiques, pénalisation des achats purement matériels et introduction d'un seuil minimal de pertinence. Après correction, cette annonce obtient un score négatif et ne peut plus apparaître dans un digest. L'ancien résultat a été volontairement conservé dans l'historique du projet : il illustre qu'un système de ce type se juge sur sa boucle de correction, pas sur sa première sortie.

8.6 Évaluation du modèle local Phi-4 Mini

Critère	Observation
Décisions attendues	Les deux dossiers de test correctement classés NO_GO
Temps d'inférence (CPU)	79 à 105 secondes par dossier ; environ 241 s pour un tour complet via la passerelle
Qualité des justifications	Certaines justifications produites n'étaient pas entièrement fondées sur les sources fournies
Usage retenu	Second avis local et outil de comparaison — pas de rôle décisionnel

Deux limites de cette évaluation doivent être posées honnêtement. Un échantillon de deux dossiers de même classe ne mesure pas une précision : il valide une faisabilité et un coût d'exécution. Et l'observation de justifications partiellement inventées disqualifie ce modèle comme moteur d'analyse principal, tout en confirmant l'intérêt du dispositif de validation en aval — c'est précisément pour ce type de comportement que la persistance d'une fiche est conditionnée à des contrôles indépendants du modèle.

8.7 Résilience aux aléas externes

La dernière collecte observée pendant le stage a échoué sur un délai de connexion du portail HAICOP. Le système a réagi comme conçu : le journal de collecte enregistre l'échec avec un message concis et

sans secret, les 100 dossiers déjà présents restent intacts et disponibles, et les autres services demeurent opérationnels. L'indisponibilité de la source et la disponibilité du référentiel local sont deux questions distinctes — c'est l'un des bénéfices concrets de la persistance.

8.8 Sauvegarde et restauration vérifiées

La sauvegarde de référence rassemble les données, documents et configurations assainies, accompagnés d'un manifeste et de sommes de contrôle. L'exercice de restauration l'a rejouée dans une base PostgreSQL isolée, a comparé les compteurs des neuf tables avec l'original, puis a supprimé la base temporaire. Le rapport d'exercice conclut au succès complet. Une sauvegarde qui n'a jamais été restaurée n'est qu'une hypothèse ; celle-ci a été éprouvée.

9. Gestion du projet

9.1 Démarche incrémentale et jalons

Le projet a été conduit par tranches verticales : chaque étage n'a été entrepris qu'une fois le précédent validé par des critères définis à l'avance. Le jalon central — relier le collecteur à PostgreSQL et prouver l'idempotence par deux exécutions réelles — a été formulé avant son implémentation, transformant un objectif vague (« stocker les données ») en un test binaire.

Date	Jalon
26 juin 2026	Cadrage : sujet confirmé, objectifs pédagogiques, choix de la source publique HAICOP
7 juillet 2026	Premier proof of concept : collecte documentée et trois analyses réelles conservées
8 juillet 2026	Retour positif de l'encadrant ; axes d'exploration fixés pour la suite
20 juillet 2026	Choix de la tranche de bout en bout : collecteur → PostgreSQL → digest local
4 août 2026	Infrastructure : PostgreSQL 16, pgAdmin et volumes persistants sous Docker Compose
8 août 2026	Collecteur relié à la base ; preuve d'idempotence par double exécution réelle
10 août 2026	Traitement documentaire complet, puis analyse persistée et empreintée
11 – 17 août 2026	Automatisation n8n, worker authentifié, digest, Mailpit puis Gmail multi-destinataires
18 – 25 août 2026	OCR et conversion DOC, Telegram, serveur MCP, évaluation Phi-4 Mini, sauvegarde et exploitation
26 août 2026	Livraison : validation finale, documentation et livrables de soutenance

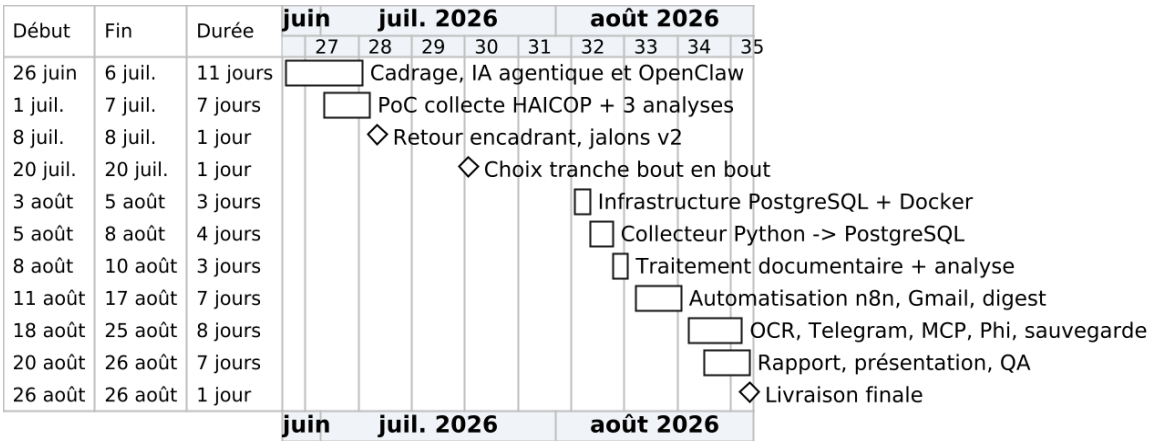


Figure 6 — Chronologie réelle du stage

9.2 Versionnement et discipline de dépôt

Le code est versionné dans un dépôt Git privé, avec des commits séparés par fonctionnalité : base, collecteur, documents, analyse, automatisation, interactions, interopérabilité, exploitation. Sont

versionnés les migrations, les tests, les prompts, les workflows et les scripts ; sont exclus les fichiers d'environnement, les données téléchargées, les contextes générés et tout secret. Les identifiants locaux de démonstration documentés dans le dépôt privé sont marqués comme devant être renouvelés avant tout partage.

9.3 Gestion des risques

Risque identifié	Réponse mise en œuvre
Indisponibilité de la source HAICOP	Timeouts, reprises, journal d'échec, référentiel local préservé
Doublons à tous les étages	Contraintes d'unicité, UPSERT, empreintes, registres de diffusion
Document malveillant ou excessif	Domaine et chemin autorisés, limites de taille, contrôle des archives, timeouts
Scans illisibles et formats anciens	OCR multilingue, conversion isolée, statuts explicites plutôt qu'échec silencieux
Hallucination du modèle	Contexte empreinté, fiche imposée, validation indépendante avant persistance
Injection d'instructions via une annonce	Textes d'appels d'offres traités comme données, jamais comme ordres
Fuite de secrets	Fichier d'environnement ignoré, erreurs expurgées, sauvegarde assainie
Perte de données	Volumes persistants, sauvegarde avec sommes de contrôle, restauration éprouvée

10. Qualité, sécurité et responsabilité

10.1 Moindre privilège

Chaque composant ne dispose que du strict nécessaire : services liés aux interfaces locales, worker joignable uniquement depuis le réseau Docker interne et n'acceptant que des routes métier authentifiées, serveur MCP limité à cinq outils bornés, prompts de l'automatisation versionnés et non modifiables par l'orchestrateur. Cette réduction systématique des surfaces vaut autant pour la sécurité que pour la prévisibilité du système.

10.2 Gestion des secrets et traçabilité

Les jetons, clés OAuth et identifiants d'infrastructure vivent exclusivement dans la configuration locale, hors du dépôt, hors des journaux et hors des sauvegardes. Côté traçabilité, tout contenu manipulé est identifiable : payloads originaux conservés, documents et contextes empreintés en SHA-256, faits officiels et raisonnement généré séparés en tables distinctes, interactions auditées par empreintes. À tout moment, on peut établir ce que le système savait lorsqu'il a produit une analyse.

10.3 Frontières légales et éthiques

Le système n'accède qu'aux ressources publiques du portail HAICOP, avec un comportement de collecte mesuré. Il ne contourne aucune authentification TUNEPS, ne réutilise aucune session et ne soumet aucune offre : lorsqu'un document relève d'un espace authentifié, la main revient à un humain autorisé. Enfin, les textes des appels d'offres sont systématiquement traités comme des données non fiables à analyser — jamais comme des instructions adressées à l'agent, ce qui constitue la défense de principe contre l'injection de prompt.

10.4 Décision assistée, jamais automatisée

La recommandation Go/No-Go a un impact commercial ; le système est donc conçu comme un instrument d'aide à la décision, pas comme un décideur. Il rassemble les preuves, rend ses critères lisibles, lie chaque fiche à l'état exact de ses sources et refuse d'enregistrer une conclusion incohérente. La validation finale appartient à l'équipe — et cette limite n'est pas une concession : c'est un choix de conception assumé.

11. Compétences développées

11.1 Compétences techniques

Domaine	Mise en pratique concrète durant le stage
Architecture logicielle	Conception par couches et par responsabilités ; frontière explicite entre déterminisme et interprétation
Python	Package structuré en onze modules, CLI JSON, API FastAPI, gestion des erreurs et des reprises
Bases de données	Modélisation relationnelle, JSONB, contraintes d'unicité, transactions, sept migrations idempotentes
Intégration HTTP	Client vérifié, pagination, timeouts, normalisation de données hétérogènes
Traitement documentaire	Extraction native, OCR Tesseract multilingue, conversion LibreOffice isolée, contrôles de sûreté
Conteneurisation	Docker Compose, volumes persistants, réseau privé, exposition minimale
Automatisation	Workflows n8n planifiés, worker authentifié, registres de diffusion idempotents
IA agentique	Skills OpenClaw, ingénierie de contexte, séparation agent/modèle/outils, évaluation d'un modèle local
Interopérabilité	Serveur MCP à cinq outils bornés sur une logique métier unique
Qualité et tests	67 tests Python, 6 scripts SQL, intégration auto-nettoyante, validations de bout en bout
Sécurité	Gestion des secrets, moindre privilège, défense anti-injection, sauvegarde et restauration éprouvées

11.2 Compétences transversales

Autonomie et prise de décision. Le projet a été conduit de bout en bout, du cadrage à la livraison. Plusieurs décisions structurantes ont été prises et assumées : isoler l'instance agentique de l'entreprise, refuser tout contournement d'authentification et concevoir en conséquence un fonctionnement où l'humain reste dans la boucle, ou encore renoncer délibérément à automatiser la décision commerciale finale.

Esprit critique. La correction du faux positif (section 8.5) illustre la posture adoptée : devant un score manifestement erroné, la réponse a été d'analyser la cause et de durcir le filtre, pas d'accepter le résultat. De même, l'évaluation du modèle local a été qualifiée pour ce qu'elle est — une mesure de faisabilité et de coût — et ses justifications partiellement inventées ont été documentées plutôt que passées sous silence.

Rigueur méthodologique. Chaque tranche du projet a été précédée de critères d'acceptation vérifiables, dont la preuve d'idempotence par double exécution réelle, définie avant l'implémentation du collecteur. Cette discipline a transformé des intentions en tests binaires.

Communication. Le travail a exigé de restituer régulièrement l'avancement à l'encadrant, de traduire des contraintes techniques en enjeux métier et de produire une documentation exploitable par un tiers : guide d'installation, procédures d'exploitation, rapports de validation et historique du projet.

Responsabilité. Les frontières légales, éthiques et de sécurité du système ont été posées explicitement et tenues tout au long du stage — jusque dans les détails, comme l'expurgation des messages d'erreur ou l'audit par empreintes des interactions de messagerie.

12. Conclusion et apports du projet

Ce stage a permis de construire, en deux mois, une chaîne complète de veille et d'analyse agentique des appels d'offres publics : collecte fiable, persistance idempotente, traitement de 152 documents hétérogènes, analyses traçables liées à leurs sources, automatisation quotidienne multicanale et interopérabilité standardisée. Les résultats du snapshot validé — 100 appels d'offres, 152 documents traités, aucun doublon observé pendant la validation, 67 tests Python et 6 scripts SQL réussis, restauration vérifiée — attestent qu'il s'agit d'un MVP local opérationnel lorsqu'il est lancé, validé et démontrable.

Au-delà des chiffres, l'apport principal du projet tient à une idée simple et exigeante : dans un système agentique, le modèle raisonne, le code vérifie et la base conserve la preuve. La séparation stricte des responsabilités — et notamment le refus de confier au modèle ce qui peut être codé, testé et contraint — rend le système auditable là où un agent monolithique resterait invérifiable.

Pour One Tech, le projet livre quatre apports concrets : une veille HAICOP centralisée, un corpus documentaire exploitable et traçable, une analyse Go/No-Go contrôlée par des règles vérifiables, et une diffusion multicanale rejouable sans renvoi doublonné observé. La documentation, les scripts d'exploitation, les tests et l'exercice de restauration constituent également un transfert technique complet : le système peut être relancé, audité et repris par un autre intervenant sans dépendre de son auteur.

Sur le plan personnel, ce stage a atteint l'objectif fixé au premier jour : comprendre ce qu'est réellement une IA agentique — non pas un modèle plus intelligent, mais un système organisé, borné et gouverné — et être capable de le concevoir, de le construire, de le valider et de l'expliquer.

Glossaire

Terme	Définition
Agent	Système qui encadre un modèle avec un objectif, un contexte et des outils bornés
AO	Appel d'offres
CLI	Interface en ligne de commande
Digest	Synthèse périodique des opportunités et de leurs analyses
Empreinte SHA-256	Somme cryptographique identifiant un contenu de manière unique
Go/No-Go	Décision d'étudier ou d'écarter une opportunité
HAICOP	Haute instance tunisienne de la commande publique ; portail public des marchés
Idempotence	Propriété d'une opération rejouable sans effet supplémentaire
JSONB	Format JSON binaire et indexable de PostgreSQL
LLM	Grand modèle de langage
MCP	Model Context Protocol, standard d'exposition d'outils aux agents
OCR	Reconnaissance optique de caractères
OpenClaw	Environnement agentique utilisé pour orchestrer les skills du projet
Prompt injection	Insertion d'instructions malveillantes dans des données lues par un modèle
Skill	Capacité d'un agent décrite en langage naturel : déclencheur, procédure, format de sortie
TUNEPS	Plateforme tunisienne d'achats publics en ligne, soumise à authentification
UPSERT	Insertion si absent, mise à jour en cas de conflit

Bibliographie et sitographie

- PostgreSQL Global Development Group, PostgreSQL 16 Documentation, postgresql.org/docs/16 — consulté en août 2026.
- Docker Inc., Docker Compose Documentation, docs.docker.com/compose — consulté en août 2026.
- Python Software Foundation, Python 3 Documentation, docs.python.org/3 — consulté en août 2026.
- Psycopg, Psycopg 3 Documentation, psycopg.org/psycopg3/docs — consulté en août 2026.
- n8n GmbH, n8n Documentation, docs.n8n.io — consulté en août 2026.
- Model Context Protocol, Specification and Documentation, modelcontextprotocol.io — consulté en août 2026.
- Ollama, Documentation, ollama.com — consulté en août 2026.
- Tesseract OCR, User Manual, tesseract-ocr.github.io — consulté en août 2026.
- The Document Foundation, LibreOffice Documentation, documentation.libreoffice.org — consulté en août 2026.
- Telegram, Bot API Reference, core.telegram.org/bots/api — consulté en août 2026.
- HAICOP, Portail des marchés publics tunisiens, marchespublics.gov.tn — consulté en août 2026.
- Documentation interne du projet : guide d'installation, historique, schéma de base, procédures d'exploitation et rapports de validation versionnés.

Annexes

Annexe A — Commandes principales du système

Démarrage et contrôle de la pile

```
.\scripts\start-onetech-stack.ps1 -StartGateway -StartOllama
.\scripts\healthcheck-onetech-stack.ps1 -RequireGateway -RequireOllama `
    -RequireWorkflows -RequireMcp
```

Collecte, traitement et consultation

```
python -m onetech_ao.cli collect --limit 50 --with-details --process-documents
python -m onetech_ao.cli status
python -m onetech_ao.cli list-open --limit 20
```

Analyse traçable d'un dossier

```
python -m onetech_ao.cli analysis-context Tender-103283
python -m onetech_ao.cli record-analysis Tender-103283 --fiche fiches\ma-
fiche.md `
    --status complete --score-adequation 20 --score-delai 18 `
    --score-administratif 15 --score-risque 17 --decision GO `
    --context-sha256 <empreinte>
python -m onetech_ao.cli show-analysis Tender-103283
python -m onetech_ao.cli digest --limit 10
```

Tests et sauvegarde

```
python -m pytest -q
.\scripts\test-database.ps1
.\scripts\backup-onetech-stack.ps1
.\scripts\test-onetech-backup-restore.ps1
```

Annexe B — Extraits SQL représentatifs

UPSERT idempotent d'un appel d'offres

```
INSERT INTO appels_offres (tender_id, organisme, titre, donnees_brutes)
VALUES (%s, %s, %s, %s)
ON CONFLICT (tender_id) DO UPDATE SET
  organisme = EXCLUDED.organisme,
  titre = EXCLUDED.titre,
  donnees_brutes = EXCLUDED.donnees_brutes,
  derniere_collecte_le = clock_timestamp();
```

Appels d'offres ouverts, par échéance

```
SELECT tender_id, organisme, titre, date_limite
FROM v_appels_offres_ouverts
ORDER BY date_limite ASC NULLS LAST;
```

Annexe C — Matrice de validation finale

Domaine vérifié	Résultat

Collecte et persistance	100 appels d'offres, aucun doublon observé lors de la validationn
Traitement documentaire	152/152 documents présents, hashés et extraits
Analyse et scoring	Fiches persistées, contrôles de cohérence actifs, replays sans doublon
Automatisation quotidienne	Workflow n8n actif et rejouable
Diffusion Gmail	Multi-destinataires avec registre anti-doublon
Diffusion et interactions Telegram	Publication réelle et réponse à une mention validées
Interopérabilité MCP	5/5 outils découverts et exécutés
Sauvegarde / restauration	Exercice complet réussi, sommes de contrôle vérifiées
Tests automatisés	67 tests Python et 6 scripts SQL réussis